https://gemini.google.com/app/e4661d1169aef631?hl=pl

Zaawansowana Refaktoryzacja Shadera Wody w Unity: Cyfrowa Integracja Estetyki Ukiyo-e z Technikami Głębi Sceny w Języku HLSL

Wprowadzenie i Kontekst Artystyczno-Technologiczny

Modelowanie oraz renderowanie powierzchni wodnych w czasie rzeczywistym należy do najbardziej złożonych zagadnień we współczesnej grafice komputerowej. O ile w symulacjach fotorealistycznych dąży się do wiernego odwzorowania równań Naviera-Stokesa, fizycznie poprawnych odbić światła (PBR - Physically Based Rendering) oraz dokładnych współczynników załamania światła (refrakcji), o tyle w grafice niefotorealistycznej (NPR - Non-Photorealistic Rendering) cel jest zgoła inny. W podejściu stylizowanym, systemy renderujące dążą do osiągnięcia określonej ekspresji artystycznej poprzez celowe zniekształcenie lub uproszczenie zjawisk fizycznych. Niniejszy raport stanowi wyczerpujące studium analityczne i techniczne dotyczące projektowania, optymalizacji oraz refaktoryzacji shadera wody w silniku Unity. Omawiany system ma za zadanie zintegrować technikę wyświetlania wielokątów typu low-poly, pasmowe oświetlenie (cel-shading) oraz unikalną, historyczną estetykę japońskich drzeworytów z okresu Edo, koncentrując się na sztuce Katsushiki Hokusaia.

Głównym problemem badawczym, stanowiącym genezę niniejszego opracowania, jest konflikt architektoniczny w kodzie HLSL (High-Level Shading Language), który doprowadził do wystąpienia wizualnego artefaktu zwanego "twardym odcięciem" (hard cutoff). Zjawisko to występuje na styku powierzchni wody i zanurzonych obiektów lub linii brzegowej, całkowicie niszcząc zamierzony efekt płynnego przejścia piany (Depth Fade) i poświaty brzegowej (Shore Glow). Analiza kodu źródłowego i struktury materiału wykazuje, że błąd wynika z niepoprawnej kolejności operacji matematycznych, gdzie funkcja kwantyzacji głębi (niezbędna do uzyskania efektu cel-shadingu) jest aplikowana globalnie, niszcząc dane potrzebne do interpolacji liniowych zjawisk przestrzennych. Aby skutecznie wyeliminować tę wadę i zapewnić architekturę na poziomie profesjonalnych rozwiązań komercyjnych (takich jak Stylized Water 3 [1]), konieczne jest wdrożenie metodologii dwutorowego przetwarzania głębi (Dual-Track Depth

Processing) oraz zaawansowanych technik antyaliasingu w przestrzeni ekranu za pomocą pochodnych cząstkowych.

Estetyka Ukiyo-e a Matematyka Grafiki Komputerowej

Zrozumienie, jak przenieść analogowe techniki graficzne do potoku renderowania na karcie graficznej (GPU), wymaga głębokiej dekonstrukcji procesu powstawania japońskich drzeworytów. Prace z nurtu Ukiyo-e (obrazy przepływającego świata), ze szczególnym uwzględnieniem ryciny "Wielka Fala w Kanagawie" (część cyklu "Trzydzieści sześć widoków góry Fudżi", około 1830-1832), charakteryzują się ścisłymi rygorami formalnymi, które wynikają z natury medium, jakim jest matryca drzeworytnicza.

Techniki Bokashi w Modelowaniu Cyfrowego Gradientu

Kluczową innowacją techniczną w sztuce Ukiyo-e była technika *bokashi* (japońskie ぼかし), pozwalająca na uzyskanie gradientów i wibracji kolorystycznych, które przełamywały płaskość tradycyjnego druku.[2] W procesie tym rzemieślnik nakładał farbę na zwilżony drewniany blok drukarski z różną intensywnością, co skutkowało płynnym przejściem barwnym.[2] Drukarnie, takie jak Eijudo, eksperymentowały z technikami takimi jak *fukibokashi* (ręczne nakładanie gradientu pędzlem na matrycę) oraz *ichimonji bokashi* (proste, horyzontalne przejścia), co pozwalało na stworzenie iluzji trójwymiarowej głębi oceanu i nieba.[2]

W kontekście shadera komputerowego, technika *bokashi* znajduje swoje matematyczne odzwierciedlenie w zjawisku zwanym Depth Fade (zanikaniem głębi). Jest to proces polegający na odczytywaniu odległości między pikselem powierzchni wody a pikselem obiektu podwodnego, a następnie interpolowaniu (funkcja lerp) barw od jasnej (woda płytka, piankowa) do ciemnej (głębia) na podstawie zlinearyzowanej odległości. Wymaga to danych wejściowych o absolutnej ciągłości – funkcja głębi nie może zawierać skoków ani progów. Zjawisko poświaty brzegowej (Shore Glow) jest bezpośrednią cyfrową emulacją *bokashi* – miękkim halo rozpraszającym światło wokół ciał stałych zanurzonych w wodzie, co potęguje iluzję objętości cieczy.

Morfologia Piany i Fal jako Ograniczenie Nośnika

Piana w pracach Hokusaia nie jest fotorealistyczną, dyfuzyjną chmurą bąbelków, lecz posiada wysoce rzeźbiarski, strukturalny charakter. Kształty piany (tzw. pazury fali) wynikają bezpośrednio z fizycznej niemożności wycięcia dłutem nieskończenie cienkich, rozmytych linii w twardym drewnie lub gumie. Krawędzie te musiały być ciągłe, ostre i mieć zdefiniowaną grubość, aby nie uległy zniszczeniu podczas odbijania kolejnych warstw grafiki w procesie tzw. druku redukcyjnego (reduction print).

W architekturze shadera wodnego w Unity, replikacja tej cechy wymaga zastosowania wysoce specyficznego łańcucha operacji. Piana nie może stanowić prostego gradientu; musi bazować na procedurach szumu (np. Perlin noise lub Value noise nakładanych na koordynaty świata lub współrzędne ekranu z offsetem czasowym). Aby naśladować wycięcia dłuta na krawędzi piany,

wynikowy szum łączy się z mapą głębi, a następnie poddaje progowaniu, najczęściej za pomocą wektorowej funkcji wygładzającej krokowej smoothstep(min, max, value). Jeśli jednak renderowany obszar to woda typu low-poly, gdzie wielokąty są duże, a wektory normalne dyskretne (powodując płaskie, niecieniowane ściany brył), ostre krawędzie wyliczane w shaderze ulegają silnemu aliasingowi (tzw. "ząbkowaniu" pikseli). Aby ukiyo-e wyglądało poprawnie, piana musi być twarda pod względem kształtu, lecz idealnie antyaliasowana na krawędziach pikseli, co wymaga zaangażowania pochodnych cząstkowych i matematyki wektorowej przestrzeni ekranu.

Konflikt wizualny polegający na nienaturalnym odcięciu piany wynika bezpośrednio z faktu, iż algorytmy pożądające wartości z pierwszej kategorii (Ciągłej) są siłą zasilane wartościami z kategorii czwartej (Dyskretnej), co stanowi fatalny błąd przepływu sterowania (control flow) w kodzie shadera.

Architektura Profesjonalnych Systemów Wody: Studium Przypadku Stylized Water 3

Aby w pełni zrozumieć, jak poprawnie zrefaktoryzować badany kod, niezbędne jest przeprowadzenie gruntownej inżynierii wstecznej rozwiązań wiodących na rynku, w tym wypadku pakietu Stylized Water 3 (SW3) stworzonego przez Staggart Creations.[1] System ten jest powszechnie uznawany za standard de facto do renderowania wody niefotorealistycznej w ekosystemie Unity, w tym na platformach docelowych wykorzystujących najnowsze rozwiązania takie jak Render Graph w Unity 6.[1]

Przejście do Nowoczesnych Systemów Renderowania

SW3 stanowi znaczący krok naprzód w ewolucji oprogramowania, ponieważ jego kod został całkowicie napisany od nowa, by wspierać strukturę Unity Render Graph.[1] Historyczne implementacje borykały się z wąskimi gardłami wydajnościowymi wynikającymi ze stosowania pełnoekranowego post-processingu (full-screen post-processing) do generowania efektów, takich jak przejścia głębi czy rendering świata podwodnego.[4] W SW3 logika ta została przepisana w taki sposób, aby zoptymalizować pracę za pomocą SRP Batchera oraz mechanizmu GPU Resident Drawer, co radykalnie zmniejsza liczbę wywołań rysowania (drawcalls).[5] Wyklucza to również wsparcie dla starszych, nieobsługujących potoków obliczeniowych architektur, takich jak WebGL (który nie obsługuje tekstur tablicowych ani Compute Shaders).[3]

Dynamiczna Piana, Falowanie i Efekty Interaktywne

Zgodnie z udostępnioną dokumentacją, fale wodne w SW3 oparte są na wielowarstwowych falach trochoidalnych, szerzej znanych w matematyce fal morskich jako fale Gerstnera (Gerstner waves).[1] Podejście to odkształca wierzchołki (vertices) wielokątów w sposób przewidywalny i parametryczny (bazując na amplitudzie, długości i stromości) wokół ich

pierwotnego wektora normalnego.[1]

System wykorzystuje zaawansowany "Height Query System" działający z poziomu GPU, który pozwala na odczyt zniekształceń wysokości na powierzchni, do którego dołączany jest system "Dynamic Effects".[1] System ten działa w oparciu o dekalowe projekcje rzutujące wertykalnie (wzdłuż osi Y) zachowanie szlaków łodzi (wake), odbijających się kropli, ciał wodnych i samej piany brzegowej.[5] W SW3 piana falowa (wave crest foam) jest obliczana przy użyciu oddzielnego zakresu min/max połączonego z bąbelkowaniem.[1]

Rozszerzone Techniki Wizualizacji i Narzędzia Artystyczne

Analiza zaawansowanych systemów stylizowanej wody (takich jak pakiety *Toon Water* dla URP) oraz współczesnych trendów w grach "low-poly" i niefotorealistycznych ujawnia, że zrefaktoryzowana architektura nie może opierać się wyłącznie na procedurach matematycznych. Wymaga udostępnienia odpowiednich dźwigni artystycznych i systemów integracji ze światem gry:

1. **Refrakcja i Kaustyka (Refraction & Caustics):** Sam Depth Fade nie wystarcza do stworzenia iluzji przezroczystości. Refrakcja polega na pobraniu pełnoekranowej tekstury (Opaque Texture w URP) i przesunięciu jej współrzędnych UV na podstawie wektorów normalnych zniekształconych falami powierzchni.[1] Tworzy to efekt załamania światła na podwodnych obiektach (np. zniekształcenie zatopionej siatki kartograficznej). Z kolei Kaustyka (Caustics) wykorzystuje zazwyczaj oddzielną ścieżkę renderowania lub dekalową projekcję, aby rzucić animowane, świetliste wzory (często tworzone z tekstur typu Voronoi) na piasek dna morskiego, reagując na kierunek padania promieni słonecznych.[6]
2. **Sterowanie Kolorami Wierzchołków (Vertex Color Support):** W profesjonalnych środowiskach programiści oddają pełną kontrolę w ręce level designerów, pozwalając im malować zachowanie shadera za pomocą Kolorów Wierzchołków (Vertex Colors) narzędziami typu Polybrush.[1] Maski przestrzenne zawarte w wierzchołkach mogą kontrolować określone zjawiska. Na przykład czerwony kanał może wymuszać pojawienie się dodatkowej piany, zielony kanał może niwelować siłę fal (np. wokół kładek i pomostów, by zapobiec przenikaniu geometrii), a niebieski kontrolować sztuczną przezroczystość zbiornika.
3. **Wielowariantowe Tekstury Piany (Included Foam Textures):** Proceduralna matematyka oparta na fwidth zabezpiecza kontury piany brzegowej i eliminuje "ząbkowanie" (aliasing). Jednak dla specyficznych stylów artystycznych, układ cieniujący musi umożliwiać nadpisanie matematyki dedykowanymi mapami bitowymi. Pakiety dostarczają odrębne tekstury dla realistycznego oceanu, sztywnego stylu komiksowego (toon-style) oraz specyficznych przejść dla wodospadów.[1] Interpolowanie proceduralnej maski brzegowej z kafelkowanymi teksturami szumu potęguje złożoność wizualną.
4. **Integracja z Systemami Cząsteczkowymi (Particle Effects):** Rozbryzgi fal, odbicia

pocisków czy kilwater statku nie powstają w samym shaderze wody, lecz w oddzielnym systemie cząstek (Particle System).[4] Aby system ten (często bazujący na animacjach klatkowych typu "flipbook") płynnie wtapiał się w płaszczyznę wody zachowując spójność oświetlenia (toon shading, normal mapping), musi obsługiwać komunikację głębi, wymuszając by płaskie wielokąty cząsteczek przyjmowały informacje wektorów normalnych samej wody.[5]

5. **Dedykowane Inspektory i Oświetlenie Toon (Custom Inspector & Lighting):** Standardowy moduł właściwości Unity ulega przeładowaniu, stając się nieczytelnym dla grafików. Systemy stylizowane posiadają dedykowany interfejs (CustomEditor), odpowiednio grupujący kolory (płytkie, głębokie - często eksponowane jako HDR dla silnych emisji świetlnych), ustawienia wielkości fal oraz fizyki światła.[7] Bardzo istotne w nurcie low-poly (Flat shading) są zmienne: Specular Cutoff (progowanie miękkości odbicia słonecznego, pozwalające uzyskać twardy, jaskrawy refleks znany ze stylistyki cel-shading), moc ramy Fresnela (Fresnel Power) określająca rozjaśnienia na horyzoncie, oraz wygładzenie krawędzi spekularnych.[6]

Mechanika Bufora Głębi w Środowisku Unity

Kluczem do naprawy problemu braku naturalnego płynnego zanikania piany w głąb (Depth Fade) jest dogłębne zrozumienie mechanizmu, za pomocą którego współczesne karty graficzne reprezentują perspektywę przestrzenną. Odbywa się to za pośrednictwem Z-Buffera (Bufora Głębi).

Parametryzacja Przestrzeni Obserwatora (Eye Space)

Kamera w Unity rejestruje światy trójwymiarowe opierając się na macierzy projekcji perspektywicznej, która rzutuje wierzchołki trójwymiarowe na dwuwymiarowy ekran. Obszar tego widoku ograniczony jest przez płaszczyznę bliską (Near Clipping Plane) oraz płaszczyznę daleką (Far Clipping Plane). Podczas rysowania nieprzezroczystych (opaque) obiektów sceny – takich jak koryta rzek, skały, piasek – karta graficzna zapisuje głębokość rzutowanego piksela w dedykowanej teksturze zwanej w Unity _CameraDepthTexture. Zmienna ta jest niezwykle specyficzna ze względu na format zapisu.

Zależnie od użytego na urządzeniu końcowym interfejsu graficznego (np. DirectX, Vulkan, Metal, OpenGL), wartości te są rzutowane na odcinek od 0 do 1 w formacie tzw. surowej głębi (Raw Depth).[8] W przypadku nowoczesnych API używa się mechanizmu "Reversed Z", co oznacza, że wartość 1.0 znajduje się dokładnie na obiektywie kamery, a wartość 0.0 na horyzoncie. Rozkład tych wartości jest wysoce nieliniowy. Matematycznie, większość precyzji zmiennoprzecinkowej bufora przypada na ułamek metra najbliżej kamery, co drastycznie zapobiega artefaktom przenikania geometrii (z-fighting). Taka nieliniowość powoduje, że surowa głębia jest całkowicie bezużyteczna do bezpośrednich operacji wizualnych, takich jak tworzenie równomiernych gradientów piany.

Linearyzacja Matematyczna Bufora Z

Aby wykorzystać wartości z _CameraDepthTexture do fizycznej analizy i modelowania zjawisk wodnych, odczytana z bufora znormalizowana i nieliniowa zmienna musi zostać przekształcona w jednostkę odległości od oka obserwatora wyrażoną w metrach (tzw. zlinearyzowana głębia oka – Linear Eye Depth).[9] W języku HLSL, powszechnie stosowaną metodą jest użycie dostarczonej przez Unity wbudowanej biblioteki przestrzeni i odwołanie się do wektora globalnego _ZBufferParams. Struktura tej parametryzacji obejmuje stałe: $x = 1 - \frac{z_{far}}{z_{near}}$ $y = \frac{z_{far}}{z_{near}}$

Wzór obliczeniowy w potoku cieniowania wykonuje następującą relację odwrotności w stosunku do parametru z_{raw} :

$$z_{linear} = \frac{1}{z_{raw} \cdot x + y}$$

Wynik operacji zwraca precyzyjną, matematycznie poprawną wartość oddalenia powierzchni podwodnej w metrach wzdłuż osi prostopadłej do kamery.[8] Ta wartość jest zawsze pozytywna.[10] W silniku Shader Graph Unity, odpowiada to nodowi "Scene Depth" ustawionemu w tryb próbkowania "Eye".[8]

Definicja Różnicy Głębi (Depth Difference / Fade Length)

Do wytworzenia jakichkolwiek organicznych efektów wody konieczne jest uzyskanie informacji o wolumenie wody ukrytej pomiędzy renderowaną płaszczyzną strumienia a dnem zbiornika. Druga zmienna, o której mowa, ukryta jest we fragmencie (pikselu) samej wody, zapisywana w macierzy Screen Position jako wartość W (tzw. screenPos.w) – zawiera ona zlinearyzowaną odległość od kamery do konkretnego trójkąta wodnego.[11]

Zatem, równanie wolumenu wody (głębokości zanurzenia danego obszaru) przybiera bardzo czytelną formację:

$$\Delta depth = \ LinearEyeDepth(SampleSceneDepth(UV)) - \ screenPos.w$$

To właśnie różnica głębi ($\Delta depth$) służy jako podstawa dla wszystkich operacji określających strefy zjawisk organicznych.

Zjawisko "Hard Cutoff" i Analiza Konfliktu Kwantyzacji (Cel-Shading)

Po wyizolowaniu zlinearyzowanej różnicy głębi ($\Delta depth$), materiał ma za zadanie wygenerować efekty optyczne. Shader analizowany objęty jest wymogiem łączenia form

low-poly z tak zwanym cel-shadingiem (toon shadingiem) i drzeworytem.[7]

Istotą cel-shadingu jest zredukowanie płynnego gradientu tonów koloru do serii dyskretnych pasm.[7] Aby uzyskać ten cel, projektant shadera aplikuje na znormalizowaną i przeliczoną głębię funkcje zaokrąglające, typowo floor() lub ceil(), wspierane mnożnikiem określającym docelową liczbę schodków.

Równanie transformacyjne kwantyzacji wygląda następująco:

$$N_{quantized} = \frac{\lfloor depth^{\gamma_{bias}} \cdot S_{steps} \rfloor}{S_{steps}}$$

Patologia przepływu (błąd "hard cutoff") powstaje dokładnie w tym miejscu. Piana (Bokashi Foam Edge) musi wyliczać swoją fizyczną grubość na podstawie ułamkowych odstępstw na krawędzi zanurzonych skał. Użycie skwantyzowanej wartości $N_{quantized}$ do ocenienia krawędzi szumu natychmiastowo ucina pianę, która sztucznie "przylepia się" do niewidocznego wielokąta cel-shadingu, zrywając z zasadami perspektywy.

Architektura Dual-Track Depth Processing

Rozwiązaniem opisanego problemu jest wdrożenie architektury Dual-Track Depth Processing (Dwutorowe Przetwarzanie Głębi), co stanowi kluczowy paradygmat nowej alokacji we Fragment Shaderze:

1. **Tor A - Strumień Ciągły (Smooth Depth):** Zawiera operacje na nienaruszonej głębi. Służy wyłącznie jako wolumetryczny nośnik informacji do określania masek przejść, refrakcji (bazującej na wektorach) i proceduralnych linii piany.
2. **Tor B - Strumień Dyskretny (Quantized Depth):** Klonuje informacje z Toru A do odizolowanego bufora wewnętrznego, a następnie poddaje zniekształceniom schodkowym. Odpowiada za cel-shading tła i ewentualny "Swamp Mode".

Tabela nowej dystrybucji logicznej dla modelu zrefaktoryzowanego:

Zjawisko / Funkcja Shadera	**Tor Poboru Zmiennej**	**Uzasadnienie Architektoniczne**
Główny Kolor Wody	Tor B (Dyskretny)	Kompresja do pasów cel-shadingu za sprawą funkcji floor.

Depth Alpha Fade	Tor A (Ciągły)	Stopniowe ujednolicenie wizualne przejścia z linią brzegu.
Krawędź Piany (Foam Edge)	Tor A (Ciągły)	Zastosowanie smoothstep i szumu wymaga gładkiego podłoża numerycznego.
Poświata Brzegu (Shore Glow)	Tor A (Ciągły)	Gładkie poszerzenie światła wokół zanurzonych głazów, przedłużające miękkość piany.

Algorytmiczna Rekonstrukcja Piany i Anti-Aliasing (fwidth)

Standardowy shader tworzy miękką, bezpostaciową pianę, co nie sprosta założeniom grubego konturu drzeworytniczego.[12] Jednak rzutowanie twardej krawędzi proceduralnej (funkcja progowa) na wielokąty niskiej gęstości powoduje silny aliasing wewnątrz samego trójkąta, którego techniki przestrzenne typu MSAA nie obejmują.

Rozwiązaniem jest wektorowa metoda antyaliasingu w przestrzeni ekranu za pomocą pochodnych cząstkowych. Oszacowanie przez GPU stopy zmian głębi pomiędzy sąsiednimi pikselami osi X i Y owocuje zmienną dynamiczną zwaną fwidth:

$$fwidth(x) = |ddx(x)| + |ddy(x)|$$

Kiedy zaimplementuje się fwidth(depthSmooth) w Torze A i podstawi wynik jako rozpiętość w interpolacji smoothstep, wymusi się sprzętową kompensację krawędzi. Szum zachowuje ostrość, a aliasing siatki zostaje wygładzony. Profesjonalne shadery chronią też wynik przed projekcją widmową (geometryczna restytucja ReconstructWorldPos z osi Y zapobiega rzutowaniu echa piany na góry w oddali pod ostrym kątem perspektywy).

Optymalizacja i Wytyczne Wydajnościowe (Performance Profiling)

Nowoczesne silniki do generowania grafiki o wysokiej wierności muszą podlegać surowym

reżimom optymalizacji [1]:

1. **Głębokość i Refrakcja:** Odczyt _CameraDepthTexture i tworzenie refrakcji podwaja w skrajnych przypadkach draw calls geometrii. Jeśli projekt docelowy bazuje na sprzęcie o ograniczonej mocy, należy przełączyć tryb głębi na uproszczony i korzystać z rzutowania Vertex Colors.[1] Zawsze wymusza się próbkowanie siatki światowym UV (Mesh UV) unikając ortogonalnego rzutowania XZ na starszych maszynach mobilnych w obawie przed błędami precyzji (floating-point jittering).[1]
2. **Ograniczanie Kosztów Kamery Planarnej:** Współpraca gęstych procedur wody (Dual-Track i refrakcja) z odbiciami (Planar Reflections) generuje potężne wąskie gardła. Refleksje winny operować z zastosowaniem precyzyjnych masek warstwowych (Culling Masks), omijając rendering nakładek (overlay renderer) dla zminimalizowania dublowania geometrii obiektywów pobocznych.[1]

Szczegółowe Instrukcje Refaktoryzacji Kodu HLSL dla Systemów Sztucznej Inteligencji

Sekcja niniejsza została sporządzona jako hermetyczna matryca instruktażowa przeznaczona bezwzględnie do skopiowania w całości i wprowadzenia do interfejsu konwersacyjnego w procesie zlecenia naprawy kodu przez LLM.

Jesteś elitarnym inżynierem układów cieniujących (Technical Shading Artist / Graphics Programmer), ekspertem we wdrażaniu rygorystycznej matematyki strumieniowej układu Unity Universal Render Pipeline (URP) przy użyciu języka HLSL (High-Level Shading Language).

Otrzymujesz do naprawy kod źródłowy shadera materiałowego dla powierzchniowej płaszczyzny wodnej (Water Shader). Zdiagnozowano w nim fundamentalny błąd w architekturze przepływu sygnału (control flow artifact): zmienna determinująca interpolację obszaru zjawisk brzegowych (m.in. pianę maskowaną przestrzennie oraz zjawisko delikatnej poświaty "Shore Glow") odwołuje się do wartości głębi (Depth), która przeszła już zniekształcenie schodkowe (kwantyzację typu floor()). Ponadto docelowy projekt uwzględnia zaawansowane parametry kontroli i efekty Low-Poly.

Jako inżynier przeprowadzisz bezwzględną restrukturyzację matematyczną opierającą się na wzorcu architektonicznym określanym jako "Dual-Track Depth Processing". Masz całkowity zakaz korzystania z funkcji opartych o step() dla generowania samej bazy szumu w celach zniwelowania aliasingu krawędzi, zachowując zarys ostry za pomocą sprzętowych pochodnych cząstkowych z wywołaniem fwidth(). Poniżej znajdują się fundamentalne kroki logiczne refaktoryzacji:

Krok 1: Wzorcowanie Perspektywy Przestrzennej Bufora Głębi

W bloku wejściowym Vertex Shader, zainicjuj procedurę ComputeScreenPos(o.vertex) przekazując wektor 4-wymiarowy (screenPos). Współrzędna w przenosi nienaruszoną,

zlinearyzowaną odległość wody z obiektywu.

We fragmencie dokonaj podziału rzutu perspektywy (uv = screenPos.xy / screenPos.w).

Odpytaj bezwzględną teksturę środowiskową (_CameraDepthTexture), a wynik zlinearyzuj:

float sceneZ = LinearEyeDepth(SampleSceneDepth(uv), _ZBufferParams);

Krok 2: Ustanowienie Architektury Dual-Track (Depth Difference)

W Torze A (Ciągłym), zaimplementuj surową różnicę odległości:

float depthSmooth = saturate((sceneZ - screenPos.w) / _Depth);

BARDZO WAŻNE: Wartość depthSmooth będzie podstawą dla zjawisk organicznych. Absolutnie zabrania się przepuszczania jej przez modyfikatory kwantyzacji typu floor.

W Torze B (Dyskretnym), przygotuj matematyczną kompresję w oparciu o Tor A:

High-level shader language

```
float depthQuantized = depthSmooth;
if (_CelEnabled > 0.5) {
   float depthPow = pow(depthSmooth, _CelBias);
   depthQuantized = floor(depthPow * _CelSteps) / _CelSteps;
}
```

Zmienna depthQuantized trafia wyłącznie do węzła odcienia akwenu: lerp(_WaterColor, _DepthColor, depthQuantized).

Krok 3: Restytucja Przestrzeni dla Piany Hokusai (Foam Edge fwidth)

Wykorzystaj depthSmooth jako nośnik maski szumowej. Zbuduj szerokość brzegową (foamEdge) na podstawie rozmiaru piany i procedury szumowej (noise * _NoiseIntensity). Użyj wbudowanej funkcji wektorowej przestrzeni ekranu zapobiegającej strzępieniu się krawędzi piany na poligonalnych ścianach:

High-level shader language

```
float rawDiff = sceneZ - screenPos.w;
float edgeWidth = fwidth(rawDiff) * 1.5;
float foamMask = 1.0 - smoothstep(foamEdge, foamEdge + edgeWidth, depthSmooth);
```

Krok 4: Emulacja Poświaty Tradycyjnej (Bokashi Shore Glow Rendering)

Poświata wnika gładko za pianą. Przesuń horyzont początkowy miękkiej aureoli w przód na bazie strumienia depthSmooth:

High-level shader language

```
float shoreOffset = max(0.0, depthSmooth - foamEdge);
float shoreGlowMask = 1.0 - smoothstep(0.0, _ShoreWidth, shoreOffset);
```

Zmieszaj efekt wynikowy do Base Color pomnażając go przez natężenie oraz gasząc wewnętrzną maską piany (1.0 - foamMask).

Krok 5: Protekcja Y-Axis (Altitude Safety Check)

Wkomponuj na początku weryfikator ReconstructWorldPos z użyciem odwrotnej macierzy kamery invViewProj. Odrzuć budowę maski foamMask, jeżeli punkt odczytany z Z-Buffera jest położony nienaturalnie wysoko względem samej tafli wody (np. odbijanie krawędzi gór w oddali).

Krok 6: Otwarta Architektura dla Narzędzi i Kanałów Wierzchołków

Zapewnij architekturze możliwość nakładania dodatkowych zachowań za pomocą wektorów wczytanych z *Vertex Colors* (np. IN.color.r dla osłabienia piany). Dołącz wsparcie dla dedykowanego inspektora materiału grupując parametry odpowiadające za cel-shading (Specular Cutoff, Fresnel Power) poprzez definicję właściwości wspieranych tagami HLSL (`` dla kolorów głębi). Udostępnij złącze wektorowe w strukturze oświetlenia pozwalające w przyszłości nadpisać współrzędne UV na potrzeby wprowadzania podwodnej refrakcji (Sample UV Offset).

Zrozumiano, zacznij refaktoryzację kodu podanego w wejściu (Input).

Podsumowanie i Wnioski Końcowe

Integracja historycznych prądów sztuki graficznej, reprezentowanej przez precyzję ramy dłuta na krawędzi matryc oraz wchłaniający proces nakładania tuszu bokashi, z matematyczną i wysoce strukturalną rygorystyką renderingu w systemie Unity, tworzy złożoną hybrydę. Zjawisko nienaturalnego odcinania stref miękkich efektów wizualnych ("hard cutoff") ujawniło nieadekwatność przepływu strumieni numerycznych w klasycznym, jednowymiarowym

paradygmacie cel-shadingu.

Analiza profesjonalnych narzędzi najwyższej wierności w gatunku (takich jak systemy Stylized Water 3 / Toon Water [1]), uzupełniona materiałami dowodowymi, dobitnie wykazała konieczność nie tylko separacji głębi logicznej (Dual-Track Depth Processing), ale również udostępnienia przestrzeni reżyserskiej w samym interfejsie. Modułowe podejście oparte na niestandardowych inspektorach, elastycznych parametrach oświetlenia (Specular Cutoff, Fresnel) oraz wsparciu barw wierzchołków (Vertex Colors), oddaje rzeczywistą moc kontroli w ręce artystów tworzących środowisko.[1] Zaimplementowana metoda fwidth() zabezpieczająca antyaliasing rzeźby piany low-poly wespół z procedurami zabezpieczeń przed widmową geometryczną dystorsją echa wysokości, składa się na projekt shadera gotowy na współczesne rygory wydajnościowe w Unity.

Cytowane prace

1. Stylized Water 3 - Documentation - Staggart Creations, otwierano: maja 10, 2026, https://staggart.xyz/unity/stylized-water-3/sw3-docs/
2. Bokashi (printing) - Wikipedia, otwierano: maja 10, 2026, https://en.wikipedia.org/wiki/Bokashi_(printing)
3. Roadmap - Staggart Creations, otwierano: maja 10, 2026, https://staggart.xyz/unity/stylized-water-2/sws-2-docs/?section=roadmap
4. Stylized Water 3 - Underwater Rendering Documentation - Staggart Creations, otwierano: maja 10, 2026, https://staggart.xyz/unity/stylized-water-3/sw3-underwater-rendering-docs/
5. Stylized Water 3 – Dynamic Effects documentation - Staggart Creations, otwierano: maja 10, 2026, https://staggart.xyz/unity/stylized-water-3/sw3-dynamic-effects-docs/
6. Stylized Water Shader (FREE!) - Page 9 - Community Showcases - Unity Discussions, otwierano: maja 10, 2026, https://discussions.unity.com/t/stylized-water-shader-free/780666?page=9
7. Stylized Surface Shader - Flat Kit - Toon Shading and Water, otwierano: maja 10, 2026, https://flatkit.dustyroom.com/stylized-surface/
8. Scene Depth node | Shader Graph | 17.0.4 - Unity - Manual, otwierano: maja 10, 2026, https://docs.unity3d.com/Packages/com.unity.shadergraph@17.0/manual/Scene-Depth-Node.html
9. Scene Depth Node | Shader Graph | 6.9.2, otwierano: maja 10, 2026, https://docs.unity3d.com/Packages/com.unity.shadergraph@6.9/manual/Scene-Depth-Node.html
10. What is "eye space" in Unity shaders? (NB: it's not view space), otwierano: maja 10, 2026, https://discussions.unity.com/t/what-is-eye-space-in-unity-shaders-nb-its-not-view-space/770029
11. I made a beginner tutorial about the depth buffer and depth texture for Shader Graph, otwierano: maja 10, 2026,

https://www.reddit.com/r/Unity3D/comments/18mv7ip/i_made_a_beginner_tutorial_about_the_depth_buffer/
12. The Great Wave: Anatomy of an Icon - The Metropolitan Museum of Art, otwierano: maja 10, 2026, https://www.metmuseum.org/essays/hokusai-great-wave